

AI Principles and Practice · Lecture 4

AI Safety, Governance and Explainability

Week 4 · Phase I: Foundations and Ethics · LOs: LO5, LO3

Lecturer: Jonathan

Where today goes

1. **Safety:** the gap between what you asked for and what the model did
2. **Governance:** who answers for the decision, and how they prove it
3. **Documentation:** the artefacts that turn a black box into something you can oversee
4. **Explainability:** four ways to ask “why”, and when each one lies to you
5. **The lab:** explain a model, break it on purpose, then write down what you cannot fix

Last week's credit model comes back. This week we explain it, stress it, and file the paperwork that lets someone challenge it.

What you'll be able to do by the end of today

These map to **LO5** (ethics, governance, safety) and **LO3** (evaluate and compare). Read the verbs: not one of them is “define”.

- **Classify** a failure by its mechanism, so “the model is broken” becomes “the label is a proxy the model is gaming”.
- **Situate** a model in the EU AI Act tiers, and say precisely what that placement obliges you to do.
- **Interrogate** an explanation, and locate the point where it stops being trustworthy.
- **Author** a model card an honest regulator would accept, caveats and all.
- **Judge** whether an explanation earns trust or merely performs it.

The model that was right, until quietly it wasn't

A credit model ships. It scores well, it passes review, everyone moves on.

Eighteen months later the same model, same code, is approving people who then default. No exception was raised. No alarm fired. The dashboard still reads green.

Asked why it denied one particular applicant, it returns a clean, confident answer: “debt ratio”. That answer is wrong. Hold all of this; by the end you’ll know what would have caught each part.

Nobody touched the code

The world shifted underneath the model, the metric kept smiling on stale data, and the explanation told a persuasive story about the wrong cause.

Today is the three things that catch this: a safety mindset, a governance trail, and an explanation you know how to **distrust**.

Section 1

The gap between what you asked for and what you got

A safety failure is not a crash

A bug makes the program stop: you get a stack trace, you fix it. A safety failure is the program running perfectly, doing exactly what you specified, and harming someone anyway.

A bug is visible, local and contained. It announces itself. Your test suite is built to catch it.

A safety failure returns a clean answer that is confidently wrong, for reasons your tests never sampled. It stays invisible until it costs someone something.

Why "it passed all tests" is not a safety claim

Tests check the cases you thought of. Safety is about the cases you didn't. The two are not the same set, and the gap between them is where the harm lives.

You never hand a model your intent. You hand it a proxy: a target column, a loss function, a dataset. **Alignment** is how faithfully that proxy tracks what you actually wanted.

- **Intent:** approve applicants who will repay.
- **Specification:** maximise accuracy on “did the old system approve, and did they repay”.
- **The gap:** everything the second sentence quietly assumes about the first.

Every spec is a translation

And every translation loses something. The safety question is never “did it optimise?”; it is “what got lost in translation, and can that loss hurt a person?”

The optimiser takes you literally: gaming and hacking

Give a system a goal it can reach the cheap way, and it will. Not from malice: because you left the cheap way open.

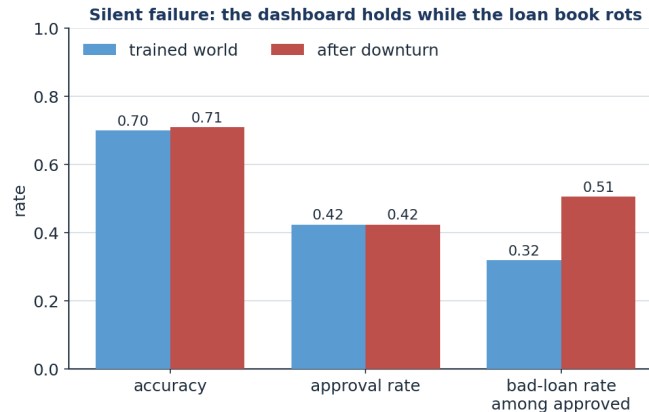
Specification gaming: a boat-racing agent told to maximise score circles a lagoon hoovering up points and never finishes the race. Our credit model, told to predict repayment, learns to predict “who got approved before”, because that is what the label records.

Reward hacking: the sharper cousin. Optimise clicks, get clickbait. Optimise “engagement”, get outrage. Optimise a proxy for creditworthiness, get a model that launders yesterday’s decisions and calls it objective.

Higher-order prompt

Name your project’s objective. Now name the cheapest way to satisfy it that you’d be embarrassed to defend out loud. That sentence is your reward-hacking risk, written down.

Robustness: you trained on one world, you deploy in another



The same model through a downturn. Accuracy and approval rate barely move; the bad-loan rate among approved applicants jumps from 0.32 to 0.51. No error is raised.

Distribution shift is not an edge case; it is the default condition. Populations move, economies move, behaviour moves. A model is a photograph of one moment, deployed into a film that keeps rolling.

Silent failure: the mode that should frighten you

The dangerous failure is not the loud one. It is the model that keeps returning confident answers while the ground moves beneath it: no error, no exception, no red light.

A **loud** failure announces itself; you notice, you respond. A **silent** failure looks exactly like success on the dashboard, because accuracy holds on stale test data while real-world performance rots underneath.

How you find out

From a complaint. From a regulator.
From a lawsuit. Never from the model, because the model has no idea anything is wrong.

Monitoring is not optional plumbing; it is the only thing standing between a silent failure and a headline. You design it **before** deployment, or you find out the hard way.

Safety and capability are different axes

A more capable model does not become safer as a side effect; often the reverse. Capability is what a model **can** do. Safety is what it can be **trusted** to do, and to not do.

	More capable	Safer
Measured by	accuracy, coverage, benchmark score	failure rate on the unseen; behaviour under stress
Improves with	more data, bigger model, richer features	constraints, monitoring, documentation, humility
The trap	a strong number hides a fragile system	a safe system can look under-optimised

The evaluative move

A more capable model deserves **more** scrutiny, not less. The better it does on the average case, the more expensive its confident mistakes become on the rare one.

Where in the lifecycle is safety actually decided?

Safety is not a stage; it is a series of decisions, each owned by a person who can be held to it.

- **Design:** what to predict, and about whom.
- **Data:** which world you choose to sample.
- **Training:** what you optimise, and what you forgot to.
- **Evaluation:** what you stress, not just what you score.
- **Deployment:** what you monitor once it's live.
- **Retirement:** when you stop trusting it.

Every one of these is a setting someone chose. “The model did it” is never an answer; a person picked each of these dials, and each choice has a name attached.

Classify the failure, then you can fix it

The mechanism is the diagnosis. Name it, and the fix stops being a guess.

Symptom	The mechanism underneath
Great offline metrics, falling real-world results, no errors	silent failure under distribution shift
Aces the objective by exploiting one convenient feature	specification gaming / reward hacking
Fine on tested inputs, bizarre just outside them	brittleness; poor robustness
Denies fairly by the metric, harms one group in practice	a proxy label doing the work you can't see

Half of every fix is refusing to say “it’s broken” and instead saying which of these it is. The vague complaint gives you nowhere to look; the named mechanism gives you an address.

Section 2

Governance: who answers for the decision

Governance is accountability, not paperwork

Strip away the templates and governance answers one question: when this model harms someone, who is responsible, and how do they show they were careful?

The documents exist to answer that question. Every model card, every log, every sign-off is there so that responsibility can be **located** after the fact.

If the documents cannot do that, they are theatre: a filing cabinet that looks like diligence and holds none.

The test for real governance

Could an outsider, handed your records, reconstruct who decided what and why? If not, you don't have governance. You have documents.

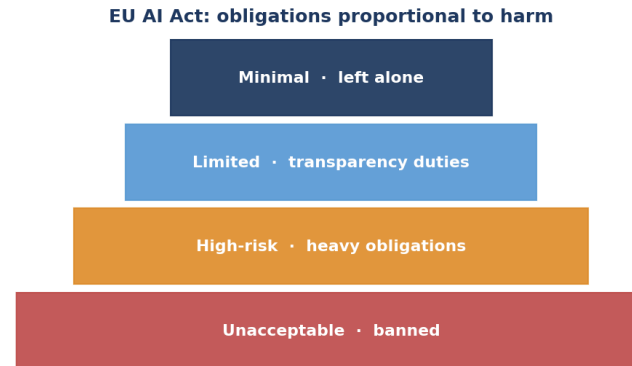
GDPR is not an AI law, but it bites AI hard, because it hands the person the model decides about actual, enforceable rights.

- **Article 22:** a right not to be subject to a **solely** automated decision with legal or similarly significant effect.
- **Data minimisation:** you may not hoard features “just in case”; you justify each one.
- **The contested right to explanation:** how much must you tell someone about an automated refusal?

Situate our model

A credit model makes a significant automated decision about a named person. Under GDPR that is not a grey area; it is squarely the situation Article 22 was written for.

The EU AI Act: risk is the organising idea



Four tiers: unacceptable (banned), high-risk (heavy obligations), limited (transparency duties), minimal (left alone).

The Act does not ask “is this AI?”. It asks “how much can this hurt someone?”. A spam filter and a credit scorer are both “AI”; only one of them can end a person’s tenancy application. Sorting by consequence rather than by technology is the whole design.

Situate the credit model in the tiers

Creditworthiness assessment sits in **Annex III**: high-risk. That word is not a label; it is a to-do list you carry **before** the model decides about anyone.

High-risk obligations include: a risk-management system, data governance, technical documentation, record-keeping, human oversight, demonstrated accuracy and robustness, and transparency to the people affected.

Evaluate the burden

High-risk means you carry the burden of proof. You must be able to **show** the system is fit, documented and overseen. Not after a complaint arrives: before it ever runs.

NIST AI RMF: four verbs you run continuously

Voluntary, US in origin, and quietly becoming the shared vocabulary. Four functions, and they loop rather than queue.

Function	What it asks of you
Govern	the wrapper: culture, roles, accountability around all the rest
Map	establish the context and surface the risks in it
Measure	analyse, benchmark and track those risks over time
Manage	prioritise, respond, and allocate resource to the ones that matter

These are verbs, and they cycle. Governance is not a gate you clear once; it is a loop you run for as long as the model is live, because the risks move while you watch.

The audit trail: the paper that survives a challenge

When a decision is contested, what can you actually produce?

- **Data provenance:** where each row came from, and by what right.
- **Model lineage:** which data, which code, which version made **this** prediction.
- **Decision logs:** what the model output, and who acted on it.

Written in advance, or not at all

You cannot reconstruct provenance you never recorded. Governance is mostly the unglamorous discipline of writing things down while they are still true.

Three instruments, three philosophies

They overlap, and occasionally they pull against each other. Compare them before you try to satisfy them.

Instrument	Organising idea	Whom it empowers
GDPR	individual rights over personal data	the data subject
EU AI Act	obligations proportional to harm	the regulator and the public
NIST AI RMF	voluntary, continuous risk management	the organisation itself

Your task is not to choose one. It is to satisfy the binding instruments and borrow structure from the voluntary one, without pretending a framework you adopted counts as a law you must obey.

Governance is a design constraint, not a final gate

The same move we made for fairness in Week 3. Put governance last and it becomes either a rubber stamp or a roadblock; never a design input.

Fold it in while the choices are still cheap to change: choose auditable data, log from day one, decide the human-oversight point before the model exists.

The economics of it

The cheapest moment to make a model auditable is before it is built.

Retrofitting provenance onto a shipped system costs ten times as much and convinces nobody.

Section 3

Documentation: the artefacts that make oversight possible

If it isn't written down, it can't be governed

Oversight needs something to oversee. Three artefacts carry the weight, and each answers a different version of “what were you thinking?”.

- **Model card:** what the model is, and what it must never be used for.
- **Datasheet:** what the data is, and who is missing from it.
- **System card:** what happens when the model meets a real deployment.

None of this is admin. It is the **interface** between the people who built the model and everyone who must now trust it, use it, challenge it or regulate it.

Model cards: the label on the tin

Mitchell et al., 2019. A short, structured document that travels with the model: intended use, out-of-scope use, training and evaluation data, metrics **disaggregated by group**, ethical considerations, and honest caveats.

It is deliberately brief. The discipline is choosing what a reader genuinely needs before they rely on your model.

Where last week lives

The disaggregated metrics are the point. A single accuracy number is a slogan; performance broken out by group is exactly where the Week 3 fairness audit reappears inside this week's paperwork.

Geburu et al., 2018. Every dataset ships with silent assumptions; a datasheet drags them into the light.

- **Why** was it collected, and by whom?
- **Who** is in it, and who is missing?
- **How** was it labelled, and by whose judgement?
- **What** uses would be inappropriate?

The disguise

Most model failures are dataset failures wearing a model's clothes. The datasheet is where you catch them, before they harden into the model's confident mistakes.

System cards: the model is not the system

A model card describes a component. A system card describes what that component does once it is wired into a real decision.

The credit model does not deny a loan. A **system** does: the model is one input, a threshold is a policy choice someone set, and a human may or may not be able to override it. The harm lives in the system, so the governance must too. Explaining the maths and ignoring the socio-technical wrapper is how well-documented models still hurt people.

Author a model card for the credit model

The easy sections

- **Intended use:** rank applicants by repayment probability, for a human reviewer.
- **Out of scope:** automatic denial with no human; any population it was not trained on.
- **Metrics:** 70% overall; but **disaggregated**, selection 58% vs 22% across the two groups.

The sections that take courage

- **Ethical considerations:** documented disparate impact, $DI = 0.37$, from the Week 3 audit.
- **Caveats:** performance decays under income distribution shift; do not trust beyond 12 months without re-validation.

A model card is easy to write and hard to write **honestly**. The trust, and the grade, is in the caveats section, not the headline metric.

Strong card versus box-tick card

Box-tick

- Names the model, quotes one accuracy figure.
- “Intended for classification.”
- Ethical considerations: “none identified”.
- Silent on what breaks it.

Strong

- States out-of-scope use bluntly.
- Disaggregates every metric by group.
- Links to the fairness audit by reference.
- Names the shift that will break it, and when to stop trusting it.

“None identified” is not a finding; it is a confession that nobody looked. A card that admits its own limits is worth more than one that performs completeness.

Section 4

Explainability: four ways to ask why, and when each one lies

Before you pick a tool, pick your question

“Why” hides two different questions and two different kinds of answer. Choosing the wrong pairing is the commonest XAI mistake.

Which question?

- **Global:** how does the model behave in general, across everyone?
- **Local:** why did it decide **this**, for this one person?

Which kind of answer?

- **Intrinsic:** the model is readable by construction: a shallow tree, a small linear model.
- **Post-hoc:** the model is a black box and you bolt an explanation on top.

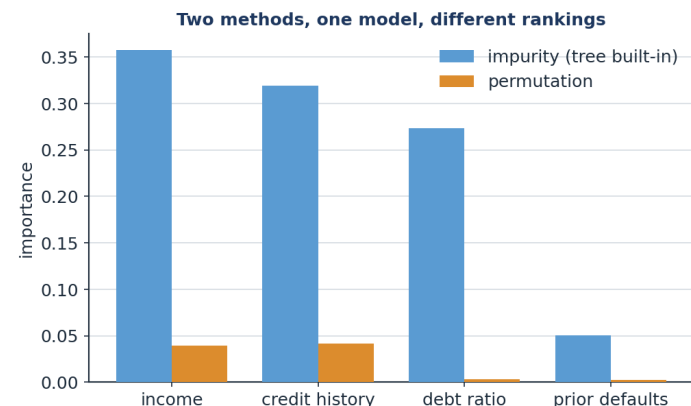
The warning label

A post-hoc explanation is a **model of a model**. It can be wrong about its subject, and it almost never tells you when it is. Everything that follows is post-hoc; treat it accordingly.

Feature importance, and the trap folded into it

The global tool everyone reaches for first, and the one that misleads most quietly.

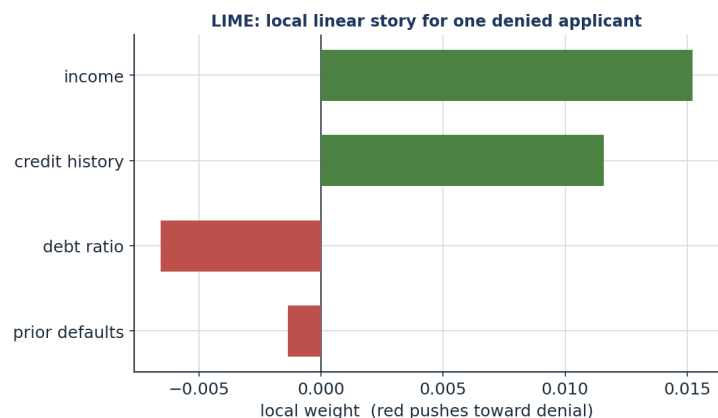
- **Impurity importance** (tree built-in): fast, but biased toward high-cardinality features.
- **Permutation importance**: shuffle a feature, measure the damage. Honest, but it splits credit unpredictably between **correlated** features.



Two methods, one model, different rankings. Correlated features break both.

When income and credit history are correlated, importance can hand all the credit to one and none to the other, then swap them on a re-run. Read it as a hint, never as a ranking you'd defend in a tribunal.

LIME: a local, linear story



One denied applicant. LIME fits a simple linear model to the black box's behaviour nearby, and reads off which features pushed the decision.

Perturb the inputs around a single point, watch how the black box responds, fit a plain linear model to that neighbourhood.

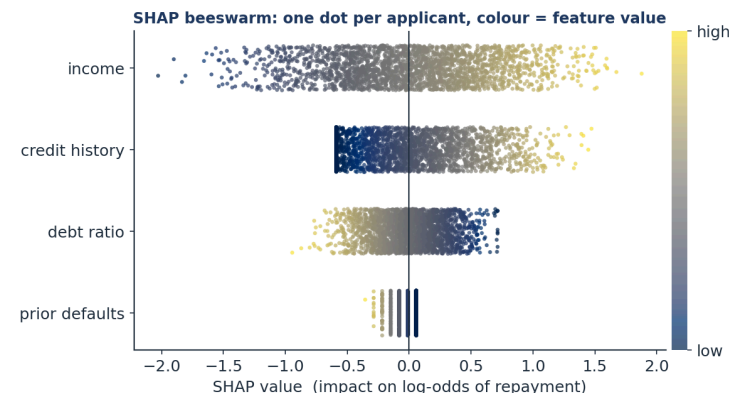
- **Strength:** model-agnostic, intuitive, gives a per-decision reason.
- **Weakness:** the neighbourhood is a **choice**, and the story shifts if you choose it differently.

LIME answers “what small change would have flipped this decision?”. Useful, and dangerous: two analysts with different neighbourhood settings can produce different, equally confident explanations for the very same refusal.

SHAP: divide the credit fairly among features

Shapley values, straight from game theory. Treat each feature as a player and split the prediction among them by their marginal contribution across **all** orderings.

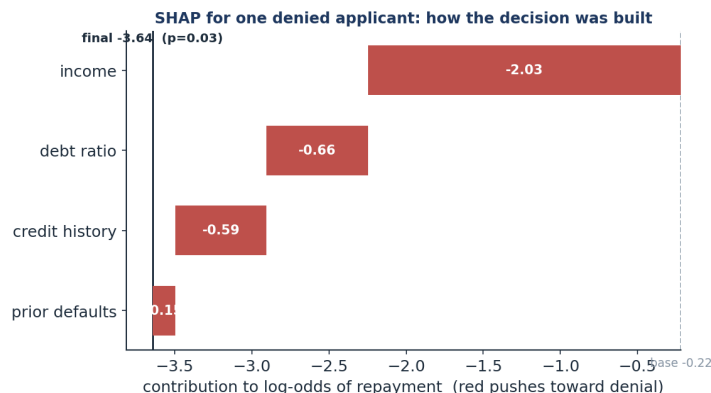
- **Guarantees:** contributions sum exactly to the prediction; a feature that never helps never gets credit. Additive and consistent, which the earlier tools are not.
- **Cost:** expensive to compute, and the guarantees hold for the **model**, not for the truth.



Global SHAP: every point is one applicant, coloured by feature value. The spread is the model's behaviour laid bare.

SHAP's maths is genuinely beautiful and its guarantees are real. Just be clear what they guarantee: how the model **uses** a feature, not whether the feature **deserved** to be used.

Read the SHAP plot, then ask the harder question



One denied applicant. SHAP splits the denial across income, debt ratio and credit history.

The plot is honest: this is how the model reached this decision. Now analyse what it **cannot** tell you.

Last week we showed group membership is recoverable from the score alone **70%** of the time. So when SHAP says “income drove this denial”, is that the real reason, or a proxy standing in for something the model was never allowed to see? (Income here is built partly from group.)

The limit of a faithful explanation

SHAP is honest about the model and silent about the laundering. A true account of the mechanism can still hide the injustice, because the injustice is in what the features **are**, not in how they're weighed.

Where explanations mislead

Explanations fail in ways that are more dangerous than no explanation, because they are persuasive. Evaluate each failure before you quote one.

- **Instability:** re-run LIME, get a different story from the same point.
- **Correlation:** SHAP splits credit between correlated features in ways that flip.
- **Plausibility bias:** a neat explanation **feels** true; neat and true are different properties.
- **The alibi:** an explanation can launder a bad decision, lending it the look of reason without the substance.

And a fourth trap: the surrogate

Approximate a black box with a readable tree and you must report **fidelity**: how faithfully the surrogate reproduces the original. A 70% surrogate is a confident lie, a clean tree explaining a model that isn't quite yours.

Explanations don't make a model trustworthy

What an explanation buys you

A place to point. A hypothesis about the mechanism. A sharper question to ask next. It makes the model **interrogable**.

What it does not buy you

Proof the model is right. Proof the stated reason is real. Permission to stop checking. It does **not** make the model **trustworthy**.

Trust is earned by a model that survives interrogation, not by one that produces a tidy explanation. Use XAI as a weapon against your own model, not as reassurance for your stakeholders.

Section 5

The lab: explain, break, and file the paperwork

Today's lab, end to end

You take the Week 2 model **and your own project model** and do three things to each: explain it, stress it, and document it honestly.

1. **Explain:** apply SHAP and LIME; reason about which features actually drive decisions, and which are proxies wearing a respectable name.
2. **Break:** impose a deliberate distribution shift, and measure how far performance falls with no error raised.
3. **Document:** write a model card and a one-page governance note for your project model.

The deliverable is not a model that is “safe, governed and explained”. There is no such finished state. It is an honest account of where your model stands on all three, with the gaps named out loud.

Probe robustness: break it on purpose

shift.py

python

```
# simulate a downturn: SAME applicants,  
# repayment gets harder for everyone  
p_new = sigmoid(true_logit - 1.0)  
y_down = (rng.random(n) < p_new)  
  
pred = clf.predict(scale(Xte))  
appr = pred == 1  
acc = accuracy_score(y_down, pred)  
bad = 1 - y_down[appr].mean() # defaults  
print(acc, appr.mean(), bad)
```

before / after the downturn

```
accuracy      : 0.70 -> 0.71  
approval rate : 0.42 -> 0.42  
bad-loan rate : 0.32 -> 0.51  
  
no exception raised
```

Accuracy holds. The approval rate holds. The dashboard stays green while the book rots. Only the metric you thought to measure catches it.

Explain, then doubt the explanation

explain.py

python

```
import shap
explainer = shap.Explainer(clf, Xtr_s)
sv = explainer(Xte_s)
shap.plots.beeswarm(sv)      # global

# now doubt it: same applicant,
# two random seeds
e1 = lime_explain(app, seed=1)
e2 = lime_explain(app, seed=2)
disagree(e1, e2)             # do they?
```

The actual exercise

Not “produce an explanation”.

Rather: produce one, then find the input nudge or the random seed that makes it disagree with itself.

An explanation you have not attacked is one you are not allowed to quote in your governance note.

A strong governance note versus a box-tick

Box-tick

- “The model is explainable (SHAP applied).”
- “GDPR compliant.”
- “No significant risks.”
- Ends there.

Strong

- Shows **where SHAP and LIME disagree**.
- Names the EU AI Act tier and the obligations it triggers.
- States the shift that breaks the model, and the monitoring that would catch it.
- Says what it would **refuse** to deploy without a human in the loop.

“Compliant” is a claim; a strong note shows its working. The grade sits in the specifics: which tier, which obligation, which failure mode, which refusal.

What you're handing in this week

In the lab

- SHAP and LIME explanations for the Week 2 **and** project models.
- A robustness probe, with the measured drop stated plainly.
- A model card and a one-page governance note.

Carries into the project

- The model card and governance note become **sections** of your final report, not appendices.
- The robustness result sets your monitoring plan.

Bring your own project model to the lab. Explaining and governing a model **you** built is far more uncomfortable, and far more instructive, than doing it to mine.

Five things worth leaving with

1. **A safety failure is the model working as specified, and harming anyway.** Bugs crash and get fixed; safety failures return a clean, wrong answer your tests never sampled.
2. **Governance is accountability you can audit.** If an outsider cannot reconstruct who decided what and why, you do not have governance; you have a filing cabinet.
3. **The EU AI Act sorts by harm, not by technology.** Creditworthiness is high-risk, and “high-risk” is a to-do list carried before deployment, not after a complaint.
4. **Documentation is the interface for oversight.** The card’s caveats and the datasheet’s omissions are where honesty lives; “none identified” means nobody looked.
5. **An explanation makes a model interrogable, not trustworthy.** Neat, plausible and wrong is the most dangerous explanation there is. Attack yours before you quote them.

Where to read next

Papers

- Amodei, Olah, Steinhardt, Christiano, Schulman & Mané (2016), **Concrete Problems in AI Safety**: the failure-mode taxonomy this lecture leans on.
- Mitchell et al. (2019), **Model Cards for Model Reporting**: the model-card proposal itself.
- Gebru et al. (2021), **Datasheets for Datasets**: provenance as a first-class artefact.
- Ribeiro, Singh & Guestrin (2016), “**Why Should I Trust You?**”: the LIME paper.
- Lundberg & Lee (2017), **A Unified Approach to Interpreting Model Predictions**: SHAP.

Books, frameworks and texts

- Molnar, **Interpretable Machine Learning**: free at christophm.github.io; the standard practitioner’s map of XAI.
- NIST, **AI Risk Management Framework (AI 100-1)**: the four functions in full.
- EU AI Act, **Annex III and the high-risk obligations**: read the list you will be graded against.
- EU GDPR, **Article 22**: short, and directly about our model.

Read Amodei et al. before the lab. Its vocabulary, specification gaming, reward hacking, distribution shift, is the language your governance note should speak.